IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Aclaraciones del reto de S04

Herramientas internas: qué se pide, cómo seguir cada tarjeta sin perderse y qué hacer cuando algo falla

Sprint 4 · Para consultar durante el reto · sin cuaderno

EL RETO · QUÉ SE PIDE

Una herramienta que calcula los indicadores de TurisData

Cliente

TurisData, de alojamientos.

Problema

Hoy los calcula **a mano**, en una hoja.

Entregable

kpis.py: **un solo comando**, **tres indicadores**.

EL COMANDO

```
python kpis.py reservas_turisdata.csv salida.json --capacidad 40
```

Los tres indicadores

Tasa de **cancelación por canal**.

Ingreso medio por reserva.

Ocupación aproximada (proxy).

Lo que sale

Un fichero **salida.json**.

Un mensaje **OK: ...** y código de salida **0**.

Si algo falla: **ERROR: ...** y código **1**, sin **Traceback**.

EL RETO · EL CAMINO DE LOS DATOS

Cuatro piezas, cada una en su misión

Orden	Pieza	Misión	Qué hace
1	`leer_reservas` (usa `_convertir_fila`)	4	Lee el CSV, pasa el texto a números y fechas y **descarta las filas malas avisando**
2	`calcular_kpis` (ya viene hecha)	3	Reúne los tres indicadores: las funciones que calculan son de la misión 3
3	`exportar_json`	5	Guarda el resultado en un fichero JSON legible
4	`main`	5	Recibe los argumentos, llama a todo **en orden** y decide el código de salida

`main` va la última: necesita todas las demás. Las misiones 0, 1 y 2 no añaden nada a `kpis.py`: preparan lo que se usa después.

Las funciones de la misión 3 trabajan con **listas en memoria, sin ficheros**: se pueden probar solas, antes de saber leer el CSV.

EL RETO · EL MAPA DE KPIS.PY

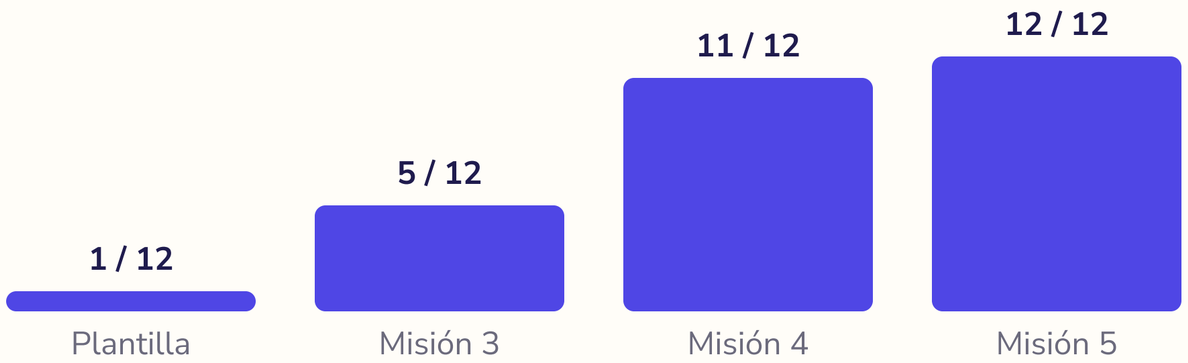
Unas 83 líneas escritas por vosotros

Función	Misión	Líneas	Recibe	Devuelve
tasa_cancelacion_por_canal	3	11	lista de reservas	diccionario **{canal: %}**
ingreso_medio_por_reserva	3	7	lista de reservas	un número (€)
ocupacion_proxy	3	13	reservas y habitaciones	un número (%)
calcular_kpis	ya hecha	5	reservas y habitaciones	diccionario con los 3 indicadores
_convertir_fila	4	15	una fila de **texto**	diccionario con tipos buenos
leer_reservas	4	19	la ruta del CSV	**(reservas, avisos)**
exportar_json	5	2	datos y una ruta	nada: escribe el fichero
main	5	16	los argumentos de la terminal	código de salida 0 o 1

leer_reservas es la única función que **devuelve dos cosas a la vez:** las reservas buenas y los avisos de las descartadas.

EL RETO · EL MARCADOR DE LAS PRUEBAS

python test_kpis.py: de 1 a 12 pruebas superadas



Cómo se lee

Cada misión **sube el marcador**.

La plantilla ya marca **1**: es la prueba opcional de pandas, que se omite y cuenta como superada.

Si el marcador no sube tras la misión, **no sigáis**: mirad el mensaje de la prueba que falla.

Se ejecuta	Cuándo	Qué comprueba
python test_kpis.py 1	Tras la misión 3 (fase 1)	Las funciones que calculan, con datos en memoria
python test_kpis.py 2	Tras las misiones 4 y 5 (fase 2)	Leer el CSV, los errores y el programa entero
python test_kpis.py	Al terminar	Las 12 pruebas, incluidos los 6 000 datos reales

ACLARACIONES · CÓMO SE SIGUE CADA TARJETA

Los pasos primero, las explicaciones después

1 · Guía paso a paso

Un **número cada vez**, de arriba abajo.

Cada paso termina en ✅: lo que **tiene que salir**.

Si no sale, **no sigáis**: id a la parte 2.

2 · Explicaciones

Qué hace cada línea.

Por qué se escribe así.

Vocabulario nuevo y «Si os atascáis».

Regla 1

Un número cada vez. No saltéis pasos: cada uno se apoya en el anterior.

Regla 2

Guardar antes de probar. Ctrl+S: la pestaña no puede tener el punto ●.

Regla 3

Pegar la función entera, de **def** a **raise**, encima de la original.

Primero lo intentáis; después de mirar el solucionario, lo cerráis y lo escribís sin mirar.

ACLARACIONES · LA SANGRÍA

Cada nivel más a la derecha significa «dentro de»

UN TROZO DE LA FUNCIÓN

```
def tasa_cancelacion_por_canal(reservas):   # nivel 0
    total = {}                              # nivel 1
    for r in reservas:                      # nivel 1
        canal = r["canal"]                  # nivel 2
        if r["cancelada"] == 1:             # nivel 2
            canceladas[canal] = 1           # nivel 3
```

Cómo se cuenta

Un nivel = **4 espacios** o **un Tab**.

El **def** es el **nivel 0**.

Lo que va debajo de **for** o **if** entra **un nivel más**.

return va al nivel de **for**: se ejecuta **al acabar** el bucle.

Si la sangría falla

IndentationError: unas líneas están **más dentro** o **más fuera** de lo debido.

Cómo evitarlo

No peguéis **líneas sueltas**: pegad la función entera.

Triple clic en la línea **raise** para seleccionarla **entera**.

ACLARACIONES · FALLOS DE LA TERMINAL

Lo que sale en rojo y qué hacer

Os sale	Casi siempre es que…	Qué hacer
arguments are required: entrada, salida	habéis usado el botón ▶ (Run), que ejecuta **sin argumentos**	Escribid el comando en la **terminal**
NotImplementedError con el código ya escrito	el fichero **no está guardado** (punto ● en la pestaña)	**Ctrl+S** y repetid la prueba
IndentationError	la sangría de alguna línea no coincide	Pegad la función **entera** del solucionario
SyntaxError en main	falta el try:, o el except no está al nivel del try	Comparad con los niveles del solucionario
Al crear el fichero pide «Python» o «Jupyter»	habéis usado «Nuevo archivo» del menú	Elegid **Archivo de texto** o el icono del Explorador

Un fallo no es un suspenso: es una pista. Leed la última línea del mensaje: dice qué ha pasado y en qué línea.

ACLARACIONES · LAS DOS ÚLTIMAS LÍNEAS

Se quedan como están

AL FINAL DE KPIS.PY

```
if __name__ == "__main__":
    sys.exit(main())
```

Qué hacen

Arrancan **main** al escribir **python kpis.py ...**.

sys.exit(...) convierte lo que devuelve **main** (0 o 1) en el **código de salida** de la terminal.

Son el **nivel 0** del fichero, igual que **def main**.

Qué no hay que hacer

No las borréis ni las cambiéis.

No peguéis la función **main** encima de ellas.

Cómo ver el código de salida

PowerShell: **$LASTEXITCODE**.

cmd de Windows: **echo %errorlevel%**.

Linux y Mac: **echo $?**.

ACLARACIONES · LOS TRES ERRORES A PROPÓSITO

El programa no se rompe: avisa

Caso	Comando	Sale	Código
Fichero que no existe	`python kpis.py no_existe.csv salida.json`	`ERROR: No se encuentra el fichero: no_existe.csv`	1
Falta una columna	`python kpis.py sin_col.csv salida_prueba.json`	`ERROR: Faltan columnas obligatorias: ['precio_noche']`	1
Dos filas incorrectas	`python kpis.py con_error.csv salida_prueba.json`	`OK: 5998 reservas analizadas, 2 descartadas -> salida_prueba.json`	0

Qué demuestra

Fichero o columna que faltan: **no hay nada que calcular**, el programa para con mensaje claro y código 1.

Filas malas: el programa **sigue**, las descarta y las apunta en **calidad.avisos**.

Si solo el primero sale bien

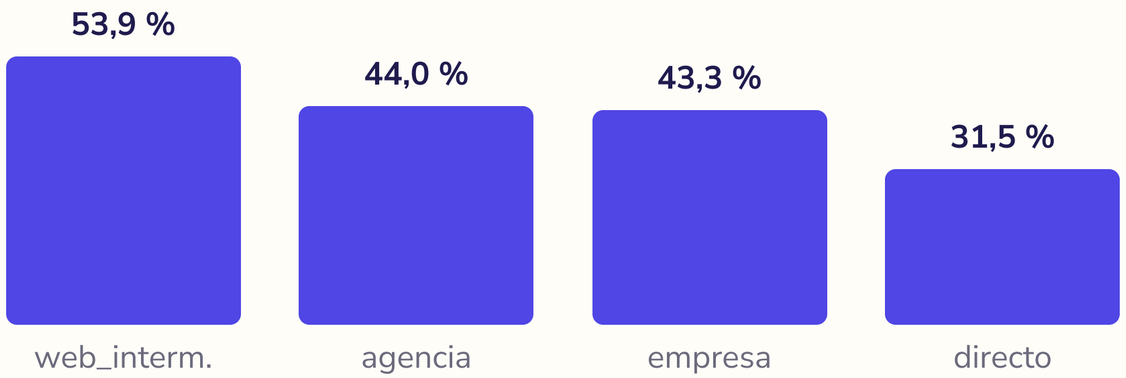
Los otros dos ficheros se editan **sobre copias** del CSV.

Comprobad que **guardasteis** esas copias (sin ●).

Descartar una fila **sin avisar** sería peor que parar.

RESULTADOS · CON EL CSV REAL

Las cifras para contrastar



Lo que sale con los 6 000 datos

6 000 reservas válidas, **0** descartadas.

3 246 confirmadas y **2 754** canceladas (**45,9 %**).

Ingreso medio: **576,04 €** por reserva confirmada.

Ocupación aproximada: **42,0 %** con 40 habitaciones.

Tasa de cancelación por canal: **canceladas del canal ÷ reservas del canal × 100**, con 1 decimal. Las canceladas **no cuentan** para el ingreso: no dejan dinero.

Si vuestra cifra no coincide, mirad de qué función sale: tasas en **tasa_cancelacion_por_canal**, ingreso en **ingreso_medio_por_reserva**, ocupación en **ocupacion_proxy**.

RESULTADOS · QUÉ DICEN Y QUÉ NO

La ocupación es un «proxy»

--capacidad	Ocupación
20 habitaciones	84,0 %
40 habitaciones	42,0 %
80 habitaciones	21,0 %

Cómo se calcula

Noches de reservas no canceladas: 17 756.

Días del periodo: 1 057 (también cuenta el primero).

100 × 17 756 ÷ (1 057 × habitaciones).

Al **doblar** la capacidad, la ocupación se queda en la **mitad**. Si no, **args.capacidad** no llega a **ocupacion_proxy**.

Lo que dicen

Casi la **mitad** de las reservas se cancela.

web_intermediaria es el canal con **más reservas** y **más cancelaciones**.

directo es el que **menos** cancela.

Lo que no dicen

Por qué se cancela más en un canal.

La **ocupación real**: no hay capacidad del hotel en el fichero.

ANTES DE ENTREGAR

Tres cosas para dar el reto por bueno

12 de 12

python test_kpis.py sin argumentos marca **12 de 12**.

3 errores controlados

Fichero que no existe, columna que falta y filas malas: **ni un Traceback**.

README con decisiones

Qué hicisteis, qué salió y **qué decidisteis** (por ejemplo, cómo definisteis la ocupación).

Lo que entreguéis, tenéis que saber explicarlo: el docente puede preguntaros por cualquier línea.